

mini-ic-flowops 完全教学手册

零基础几天上手 IC 流程自动化实习准备项目

生成日期：2026-06-08

适合对象：Windows 电脑 + Linux 虚拟机、Shell/Tcl/Python 零基础或基础较弱、即将进入 IC/EDA/CAD 流程自动化相关实习的同学。

阅读目标：读完并完成操作后，你应该能独立跑通项目，知道每个目录和命令的作用，能看日志、看报告、做 IP QA、分析测试数据，并能把项目清楚讲给导师听。

目录

本手册建议按顺序阅读。每章都围绕一个实际能力点，不要求你第一遍记住全部细节。

1. 项目和岗位到底有什么关系
2. Windows 与 Linux 虚拟机准备
3. 第一次完整跑通项目
4. 每条命令逐句解释
5. 项目目录和文件逐个讲解
6. Reference Flow 与 Testchip Flow
7. 日志、报告、metrics、manifest 怎么看
8. Shell、Tcl、Python 的分工
9. 核心代码阅读路线
10. IP QA 自动化
11. Testchip 测试数据分析
12. 版本管理与结果追溯
13. 常见报错和排查方法
14. 3 到 5 天冲刺学习计划
15. 最终自测与实习表达模板
16. 实操实验：从只会跑到能修改
17. 输出文件逐个阅读训练
18. 真实实习场景模拟
19. 代码伪代码和函数地图

第 1 章：项目和岗位到底有什么关系

岗位截图的核心不是让你马上会真实芯片设计，而是要求你能围绕内部需求完成脚本工具编写和工作环境配置。真实公司里，你常接触的是 Linux 服务器、EDA 工具、项目目录、运行日志、报告文件、版本管理、测试数据和自动化脚本。

mini-ic-flowops 是一个训练这些工作方式的模拟项目。它不使用真实 PDK 或商业 EDA 工具，而是用可学习、可运行、可修改的方式复刻岗位常见任务。

岗位要求	项目对应内容	你要学会什么
数据版本管理自动化	manifest.json、Git commit、输入文件 hash	知道一次运行为什么要可追溯
模拟电路设计流程自动化	flow 配置和步骤编排	理解自动化流程如何串起来
Reference Flow/Testchip Flow	config/ref_flow.yaml、config/testchip_flow.yaml	知道基础流程和完整流程的区别
版图设计流程自动化	floorplan/place/route/signoff Tcl 步骤	了解布局布线和签核在流程中的位置
IP QA	flowctl qa、run_ip_qa()	知道交付包检查什么
流片测试数据管理分析	data/testchip_results.csv、analyze_testchip_data()	能统计良率和失败原因
AI 工具部署支持	flowctl ask、docs/ai_tool_deploy_notes.md	理解内部知识检索和数据安全意识

现实边界：学完本项目不能保证你会真实商业 EDA 工具，但能让你具备入职初期最需要的脚本、环境、日志、报告、QA、数据分析和沟通能力。

第 2 章：Windows 与 Linux 虚拟机准备

你现在是 Windows 电脑，并且已经装了 Linux 虚拟机。建议把项目复制到 Linux 虚拟机中运行，因为 IC/EDA 公司大量使用 Linux 服务器。

推荐安装工具

Ubuntu 虚拟机中执行：

```
sudo apt update
sudo apt install -y python3 git make tcl unzip
```

Rocky Linux 或 CentOS 中执行：

```
sudo dnf install -y python3 git make tcl unzip
```

复制项目到 Linux

如果你拿到的是 zip 文件，假设它在 Downloads 目录：

```
cd ~/Downloads
unzip mini-ic-flowops_project.zip
cd mini-ic-flowops
```

确认你在项目根目录

```
pwd
ls
```

你应该能看到这些目录或文件：

```
bin config data designs docs scripts runs reports README.md START_HERE.md
```

新手重点：大部分命令失败不是代码坏了，而是你不在正确目录、没有执行 chmod、没有 source 环境，或者 Linux 里缺少 python3/tcl/git。

第 3 章：第一次完整跑通项目

第一次运行的目标只有一个：看到完整 Testchip Flow 跑通，并生成 run 目录、日志和报告。不要一开始就钻代码。

第一组命令

```
cd mini-ic-flowops
chmod +x bin/flowctl
source config/env.sh
flowctl doctor
flowctl run --design alu --flow testchip
flowctl status
```

命令执行后应该看到什么

`flowctl run` 成功后，会打印类似：

```
[OK] 运行完成：run_001
日志：runs/run_001/logs/flow.log
报告：runs/run_001/reports/flow_report.html
```

如果你的编号不是 run_001，而是 run_006 或其他编号，也正常。每运行一次都会生成新的 run 编号。

检查结果目录

```
ls runs
ls runs/run_001
ls runs/run_001/logs
ls runs/run_001/reports
ls runs/run_001/metrics
```

你今天必须记住

- `runs/run\_xxx/logs/flow.log` 是主日志。
- `runs/run\_xxx/reports/flow\_report.html` 是总报告。
- `runs/run\_xxx/metrics/\*.metrics` 是每个步骤的指标。
- `runs/run\_xxx/manifest.json` 是这次运行的版本和输入记录。

第 4 章：每条命令逐句解释

这一章把常用命令拆开讲。你不需要背，但要能看懂命令在让电脑做什么。

命令	作用	常见问题
cd mini-ic-flowops	进入项目目录	路径不对会提示 No such file or directory
chmod +x bin/flowctl	给脚本执行权限	不做可能 Permission denied
source config/env.sh	加载环境变量	每次新开终端都建议执行
flowctl doctor	检查 Python/Git/Tcl 环境	Tcl 缺失时项目仍可 fallback
flowctl run --design alu --flow testchip	运行完整流程	失败先看 flow.log
flowctl status	查看所有 run 状态	看最新 run 编号
flowctl qa --design alu	单独做 IP QA	可用于检查 IP 目录完整性
flowctl compare run_001 run_002	比较两次运行指标	编号要换成你自己的
flowctl ask "怎么看日志"	搜索 docs 文档	模拟内部知识库检索
flowctl learn	显示学习路线	零基础入口命令
flowctl archive --run latest	归档最新 run	生成 reports/run_xxx.zip

最常用的日志查看命令

```
tail -n 80 runs/run_001/logs/flow.log
```

`tail -n 80` 表示看文件最后 80 行。很多失败原因在日志末尾，所以这是非常常用的排查动作。

第 5 章：项目目录和文件逐个讲解

真实工作中，你经常不是从写代码开始，而是先进入一个陌生项目目录，理解每个文件夹的作用。

路径	作用	你要关注什么
bin/	命令入口	flowctl 如何启动
config/	流程和环境配置	ref_flow.yaml、testchip_flow.yaml、env.sh
designs/	设计输入	alu 示例 IP、RTL、SDC、ip.yaml
scripts/shell/	Shell 小工具	查看日志等小自动化
scripts/tcl/	Tcl 模拟 EDA 步骤	synth/floorplan/place/route/sign off
scripts/python/	Python 自动化核心	flowctl.py 是主程序
data/	测试数据	testchip_results.csv
runs/	每次运行结果	日志、报告、metrics、manifest
reports/	汇总报告和归档	run_xxx.zip、HTML 报告
docs/	学习文档	几天冲刺计划和自测清单

示例设计目录

```
designs/alu/  
  README.md  
  ip.yaml  
  rtl/alu.v  
  constraints/alu.sdc  
docs/interface.md
```

这里的 ALU 是一个简化 IP。你现在不需要精通 Verilog，只要知道 RTL 是设计输入，SDC 是约束，ip.yaml 是交付元数据。

第 6 章：Reference Flow 与 Testchip Flow

Flow 就是一串步骤。自动化的核心，是把这些步骤稳定、可配置、可追溯地跑起来。

Reference Flow

配置文件：`config/ref\_flow.yaml`。

```
# Reference Flow: 基础参考流程
name: ref
description: "基础 RTL 到 signoff 的模拟参考流程"
steps:
  - precheck
  - synth
  - floorplan
  - place
  - route
  - signoff
  - report
```

Reference Flow 是基础参考流程，适合验证一个设计能否从输入一路跑到 signoff 并生成报告。

Testchip Flow

配置文件：`config/testchip\_flow.yaml`。

```
# Testchip Flow: 在基础流程后增加 IP QA 和测试数据收集
name: testchip
description: "模拟 testchip 设计、QA、测试数据分析的完整流程"
steps:
  - precheck
  - synth
  - floorplan
  - place
  - route
  - signoff
  - ip_qa
  - data_collect
  - report
```

Testchip Flow 在基础流程后增加 IP QA 和测试数据分析，更贴近岗位截图里的完整工作。

步骤	中文理解	本项目输出
precheck	检查输入是否完整	precheck.md/json
synth	综合，RTL 到门级网表的模拟	synth.rpt、synth.metrics
floorplan	规划芯片面积和核心区域	floorplan.rpt
place	标准单元摆放	place.rpt
route	布线	route.rpt
signoff	签核，检查 timing/DRC/LVS/power	signoff.rpt

步骤	中文理解	本项目输出
ip_qa	IP 交付包质量检查	ip_qa_report.html/json/md
data_collect	测试数据统计分析	testchip_analysis.md/json
report	汇总报告	flow_report.html/md

第 7 章：日志、报告、metrics、manifest 怎么看

主日志 flow.log

主日志记录每个步骤什么时候开始、什么时候通过、是否用了 Tcl 或 fallback。

```
tail -n 80 runs/run_006/logs/flow.log
```

报告 reports

报告是给人看的。HTML 可以用浏览器打开，Markdown 和 rpt 可以用文本编辑器打开。

```
ls runs/run_006/reports
```

指标 metrics

metrics 是给脚本比较和统计的，通常是 key=value 格式。

```
cat runs/run_006/metrics/signoff.metrics
```

manifest.json

manifest 是一次运行的身份证。它记录设计名、flow 名、开始时间、结束时间、Git commit、工具版本、输入文件 hash、每个步骤状态。

```
cat runs/run_006/manifest.json
```

为什么公司很重视 manifest：因为结果变了时，必须判断是代码变了、配置变了、输入变了、工具版本变了，还是环境变了。没有 manifest，很多问题只能靠猜。

本手册生成时，最终示例 run_006 对应的 commit 是 `024082d`。

第 8 章：Shell、Tcl、Python 的分工

语言	本项目文件	主要作用	真实工作中的常见用途
Shell	bin/flowctl、config/env.sh	入口、环境变量、串命令	source 工具环境、跑 flow、查日志
Tcl	scripts/tcl/*.tcl	模拟 EDA 工具步骤	控制 DC/PT/ICC2/Innovus/Tempus 等工具
Python	scripts/python/flowops/flowctl.py	流程编排、QA、数据分析、报告	自动化平台、数据处理、报告生成

Shell 入口示例

```
#!/usr/bin/env bash
set -euo pipefail

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
export FLOWOPS_ROOT="$PROJECT_ROOT"

python3 "$PROJECT_ROOT/scripts/python/flowops/flowctl.py" "$@"
```

这段 Shell 的核心是找到项目根目录，设置 FLOWOPS_ROOT，然后把所有参数转交给 Python 主程序。

Tcl 步骤示例

```
# Tcl step: synth
set run_dir [lindex $argv 0]
set design_dir [lindex $argv 1]
set metrics_path [lindex $argv 2]
set report_path [lindex $argv 3]

set rtl_files [glob -nocomplain -directory "$design_dir/rtl" *.v]
set rtl_count [llength $rtl_files]
set cell_count [expr {120 + $rtl_count * 17}]
set area_um2 [expr {$cell_count * 6.25}]
set wns_ns 0.18

set fp [open $metrics_path w]
puts $fp "step=synth"
puts $fp "rtl_files=$rtl_count"
puts $fp "cell_count=$cell_count"
puts $fp "area_um2=$area_um2"
puts $fp "wns_ns=$wns_ns"
close $fp

set rp [open $report_path w]
puts $rp "SYNTHESIS REPORT"
puts $rp "Design dir: $design_dir"
puts $rp "RTL files: $rtl_count"
puts $rp "Cell count: $cell_count"
puts $rp "Area um2: $area_um2"
puts $rp "WNS ns: $wns_ns"
close $rp
```

```
puts "synth completed"
```

Tcl 脚本读取参数，生成 metrics 和 report。真实 EDA 工具中，这些模拟指标会被 read_verilog、compile、report_timing 等真实命令替换。

第 9 章：核心代码阅读路线

不要从第 1 行读到最后 1 行。按下面顺序读，先抓主线。

1. 读 bin/flowctl，知道命令如何进入 Python。
2. 读 config/testchip_flow.yaml，知道要跑哪些步骤。
3. 读 flowctl.py 的 build_parser()，知道有哪些子命令。
4. 读 cmd_run()，知道完整流程如何执行。
5. 读 run_tcl_step()，知道 Python 如何调用 Tcl。
6. 读 run_ip_qa()，知道 QA 怎么检查。
7. 读 analyze_testchip_data()，知道 CSV 怎么分析。
8. 读 build_report()，知道报告怎么生成。

cmd_run 的逻辑

cmd_run 是主流程。它找到设计和 flow 配置，创建 run 目录，写 manifest，循环执行每个 step，成功则记录 PASS，失败则记录 FAIL 并写日志。

build_parser 的作用

build_parser 定义命令行接口。比如 ``flowctl run`` 对应 ``cmd_run()``，``flowctl qa`` 对应 ``cmd_qa()``，``flowctl learn`` 对应 ``cmd_learn()``。

你需要理解但不用死背的 Python 概念

- 函数：一段可重复调用的逻辑。
- 字典 dict：保存 key/value，例如 metrics。
- 列表 list：保存多个步骤或多行数据。
- Path：表示文件路径。
- subprocess：Python 调用外部命令，例如 tclsh。
- json/csv/html：不同格式的数据和报告。

第 10 章：IP QA 自动化

IP QA 的目标是确认一个 IP 交付包是否完整、规范、可集成。真实项目中，如果 IP 包缺文件、缺版本、路径写死，会导致后续 flow 或集成失败。

运行命令

```
flowctl qa --design alu
```

本项目检查项

- README.md 是否存在。
- ip.yaml 是否存在。
- RTL 文件是否存在。
- SDC 约束是否存在。
- ip.yaml 是否有 version 字段。
- 是否存在空文件。
- 是否硬编码绝对路径。

QA 输出

```
reports/alu_standalone_ip_qa/ip_qa_report.html  
reports/alu_standalone_ip_qa/ip_qa_report.md  
reports/alu_standalone_ip_qa/ip_qa_report.json
```

练习：制造一个失败

```
cp -r designs/alu designs/badip  
rm designs/badip/README.md  
flowctl qa --design badip  
cp designs/alu/README.md designs/badip/README.md  
flowctl qa --design badip  
rm -rf designs/badip
```

这个练习能帮你理解 QA 不是抽象概念，而是自动检查文件和规范。

第 11 章：Testchip 测试数据分析

岗位截图提到 IP 流片测试数据收集、管理及分析。本项目用 CSV 模拟测试结果，再用 Python 统计良率和失败原因。

数据文件

```
chip_id,lot,wafer,die_x,die_y,ip_name,testcase,voltage,freq,result,fail_reason
TC001,L01,W01,0,0,alu,add_basic,0.80,100,PASS,
TC002,L01,W01,0,1,alu,sub_basic,0.80,100,PASS,
TC003,L01,W01,0,2,alu,and_or,0.80,100,PASS,
TC004,L01,W01,1,0,alu,shift_left,0.80,100,FAIL,timing_margin
TC005,L01,W01,1,1,alu,shift_right,0.80,100,PASS,
TC006,L01,W01,1,2,alu,add_basic,0.72,120,FAIL,low_voltage
TC007,L01,W02,0,0,alu,sub_basic,0.72,120,PASS,
TC008,L01,W02,0,1,alu,and_or,0.72,120,PASS,
TC009,L01,W02,0,2,alu,shift_left,0.72,120,FAIL,timing_margin
TC010,L01,W02,1,0,alu,shift_right,0.72,120,PASS,
TC011,L02,W01,0,0,timer,reset_basic,0.80,100,PASS,
TC012,L02,W01,0,1,timer,counter_rollover,0.80,100,FAIL,functional
TC013,L02,W01,0,2,timer,interrupt,0.80,100,PASS,
TC014,L02,W01,1,0,gpio,input_sample,0.80,100,PASS,
TC015,L02,W01,1,1,gpio,output_drive,0.80,100,PASS,
TC016,L02,W02,0,0,gpio,input_sample,0.72,120,PASS,
TC017,L02,W02,0,1,gpio,output_drive,0.72,120,FAIL,signal_integrity
TC018,L02,W02,0,2,alu,add_basic,0.90,150,PASS,
TC019,L02,W02,1,0,alu,shift_left,0.90,150,FAIL,timing_margin
TC020,L02,W02,1,1,timer,counter_rollover,0.90,150,PASS,
```

字段解释

字段	含义
chip_id	芯片或测试样本编号
lot	批次
wafer	晶圆编号
die_x/die_y	die 在 wafer 上的位置
ip_name	被测 IP 名称
testcase	测试用例
voltage	测试电压
freq	测试频率
result	PASS 或 FAIL
fail_reason	失败原因

分析输出

Testchip Flow 会生成：

```
runs/run_006/reports/testchip_analysis.json  
runs/run_006/reports/testchip_analysis.md
```

报告会告诉你总测试数、通过数、失败数、良率、失败最多的 IP、失败最多的 testcase、失败原因分布。

第 12 章：版本管理与结果追溯

版本管理不是为了显得专业，而是为了在结果变化时能追责和复现。

常用 Git 命令

```
git status
git log --oneline -5
git diff
```

本项目已有 commit

```
024082d Add beginner crash course docs
8b74825 Improve beginner document search
5cb0421 Initial mini IC flowops training project
```

manifest 里记录什么

- run_id: 运行编号。
- status: PASS 或 FAIL。
- design/flow: 本次跑哪个设计和流程。
- git_commit: 本次运行对应的代码版本。
- tool_versions: Python/Git/Tcl 版本。
- inputs.files: 输入文件和 sha256。
- steps: 每个步骤的状态和耗时。

实习表达：我会保留每次 flow run 的 manifest，用于追踪代码版本、输入文件和工具环境，方便后续结果比较和问题定位。

第 13 章：常见报错和排查方法

现象	最可能原因	解决方法
No such file or directory	当前目录不对	pwd/ls 确认位置，再 cd 到项目根目录
Permission denied	脚本没有执行权限	chmod +x bin/flowctl
flowctl: command not found	没有 source 环境	source config/env.sh
python3 missing	Linux 没装 Python	sudo apt install -y python3
tclsh missing	Linux 没装 Tcl	sudo apt install -y tcl；项目可先 fallback
precheck 失败	设计输入缺文件	检查 designs/alu/README.md、ip.yaml、rtl、constraints
QA FAIL	IP 包不完整	打开 ip_qa_report.html 看具体检查项
compare 找不到 run	run 编号写错	flowctl status 查编号

标准排错顺序

1. 看终端最后输出。
2. 看 runs/run_xxx/logs/flow.log。
3. 找第一个失败步骤。
4. 打开对应 report。
5. 看 manifest.json，确认输入和版本。
6. 如果还不懂，再向导师描述 run 编号、失败步骤、日志路径和第一个错误。

第 14 章：3 到 5 天冲刺学习计划

天数	目标	必须完成
Day 1	跑通项目	flowctl doctor、run testchip、status、看 flow.log
Day 2	看懂目录和 flow	读 project_file_walkthrough、flow_concepts、看 reports/metrics
Day 3	看懂代码主线	读 bin/flowctl、cmd_run、run_tcl_step、synth.tcl
Day 4	动手改和比较	改一个指标，重新 run，用 compare 比较
Day 5	自测和表达	完成 final_self_test，练习 30 秒和 1 分钟介绍

每天最少要做到

- Day 1: 不看别人操作，自己跑出 PASS。
- Day 2: 能说出 runs、reports、manifest 分别是什么。
- Day 3: 能说出 Shell/Tcl/Python 分工。
- Day 4: 能制造一个 QA 失败并修复。
- Day 5: 能把项目讲给别人听。

第 15 章：最终自测与实习表达模板

最终操作自测

```
cd mini-ic-flowops
source config/env.sh
flowctl doctor
flowctl run --design alu --flow testchip
flowctl status
flowctl qa --design alu
flowctl ask "怎么看日志"
flowctl archive --run latest
```

30 秒介绍

我做了一个 mini IC flow 自动化项目，用来训练脚本工具和流程自动化能力。它用 Shell 做统一入口，用 YAML 配置 Reference Flow 和 Testchip Flow，用 Tcl 模拟 EDA 工具步骤，用 Python 做流程编排、IP QA、测试数据分析和报告生成。每次运行都会保留日志、报告、metrics 和 manifest，方便排查失败和追踪版本。

1 分钟介绍

这个项目模拟了芯片团队内部常见的 flow 工具。用户运行 flowctl run 后，程序会读取 flow 配置，创建独立的 run 目录，然后按 precheck、synth、floorplan、place、route、signoff、ip_qa、data_collect、report 的顺序执行。Tcl 脚本负责模拟 EDA 工具报告，Python 负责调度流程、检查 IP 交付质量、分析 testchip CSV 数据，并生成 HTML/Markdown 报告。manifest.json 会记录 git commit、工具版本、输入文件 hash 和每个步骤状态，所以结果可以追溯和比较。

你不要夸大

不要说“我已经完全掌握 IC 设计流程”。更真实的说法是：我还在学习真实 EDA 工具和芯片设计细节，但我已经通过项目练过 Linux 环境、Shell/Tcl/Python 脚本、流程编排、日志排查、QA 和数据分析，能比较快接手流程自动化类任务。

第 16 章：实操实验：从只会跑到能修改

这一章是你真正建立手感的部分。只看文档会觉得懂了，但实习时需要的是能动手、能定位、能解释。每个实验都很小，但都对应真实工作里的动作。

实验 1：从零跑通完整流程

目标：确认你能独立跑出 PASS。

```
cd mini-ic-flowops
source config/env.sh
flowctl doctor
flowctl run --design alu --flow testchip
flowctl status
```

验收：`flowctl status` 中最新 run 的 Status 是 PASS。

实验 2：只跑基础 Reference Flow

目标：理解 ref flow 和 testchip flow 的区别。

```
flowctl run --design alu --flow ref
flowctl status
cat config/ref_flow.yaml
cat config/testchip_flow.yaml
```

观察：ref flow 没有 ip_qa 和 data_collect；testchip flow 有。

实验 3：查看每一步报告

目标：知道每一步输出什么。

```
ls runs/run_001/reports
cat runs/run_001/reports/synth.rpt
cat runs/run_001/reports/signoff.rpt
cat runs/run_001/reports/testchip_analysis.md
```

如果你的 run 编号不是 run_001，请用 `flowctl status` 找最新编号。

实验 4：制造 QA 失败并修复

目标：理解 IP QA 的价值。

```
cp -r designs/alu designs/badip
rm designs/badip/README.md
flowctl qa --design badip
cp designs/alu/README.md designs/badip/README.md
flowctl qa --design badip
rm -rf designs/badip
```

解释：第一次 QA 应该失败，因为 README

缺失；修复后应该通过。这就是自动化检查交付包完整性的意义。

实验 5：比较两次运行

目标：理解结果追溯。

```
flowctl run --design alu --flow ref
flowctl run --design alu --flow ref
```

```
flowctl status
flowctl compare run_001 run_002
```

注意：编号要换成你自己的。比较结果通常相同，因为输入和脚本没有变化。真实公司里，如果面积/功耗/时序变化，compare 能帮你定位差异。

实验 6：看 manifest 判断输入是否变化

```
cat runs/run_001/manifest.json
```

重点看 `git\_commit`、`tool\_versions`、`inputs.files` 和 `steps`。这四块是结果可追溯的核心。

实验 7：搜索内部文档

```
flowctl ask "怎么看日志"
flowctl ask "IP QA 检查什么"
flowctl ask "manifest 是什么"
```

解释：这模拟了内部知识库检索。真实 AI 工具部署时，第一步往往不是聊天，而是让员工能从内部文档快速找到答案。

实验 8：归档一次运行

```
flowctl archive --run latest
ls reports
```

解释：归档能把一次运行的日志、报告和 manifest 打包保存，方便交付给导师或留档。

第 17 章：输出文件逐个阅读训练

这一章教你如何从输出文件反推整个流程。实习时导师经常不会先讲所有背景，而是给你一个 run 目录，让你自己看结果。

1. flow.log

位置：`runs/run\_xxx/logs/flow.log`。这是第一优先级。

你要找的信息：

- run 是什么时候开始的。
- 每个 step 是什么时候开始和通过的。
- 是否出现 failed/error/exception。
- Tcl 是否真实执行，还是用了 Python fallback。

2. precheck.md

位置：`runs/run\_xxx/reports/precheck.md`。它告诉你输入是否完整。

如果 precheck 失败，不要先怀疑 Python/Tcl，先检查设计目录里是否缺 README、RTL、SDC 或 ip.yaml。

3. synth.rpt

综合报告。真实公司里它可能包含面积、cell

数、warning、约束情况。本项目里是模拟版本，用来训练你理解 report 的结构。

4. route.rpt

布线报告。本项目关注 routed_nets、drc_violations、antenna_violations。真实项目里 route 阶段可能产生拥塞、DRC、天线效应等问题。

5. signoff.rpt

签核报告。重点看 setup/hold、DRC、LVS、power。真实工作里 signoff 往往是最关键的质量关口。

6. ip_qa_report.html

这是 IP 交付质量报告。你要看 PASS/WARN/FAIL 的项目，以及失败原因。

7. testchip_analysis.md

这是测试数据分析报告。你要能回答：总测试数是多少，良率是多少，哪个 IP 失败最多，失败原因是什么。

8. flow_report.html

总报告。它把 metrics、QA、testchip analysis 汇总在一起。给导师看时通常先打开它。

9. manifest.json

这是机器可读的运行记录。不要嫌它长，真实工作里它非常重要。

manifest 字段	你要怎么看
status	本次 run 是 PASS 还是 FAIL
design/flow	跑的是哪个设计、哪套流程
git_commit	本次代码版本
tool_versions	Python/Git/Tcl 环境
inputs.files	输入文件和 hash
steps	每个步骤状态和耗时
error	如果失败，记录失败原因

第 18 章：真实实习场景模拟

下面这些场景比单纯背命令更接近真实实习。你可以把它们当作小型作业。

场景 1：导师让你帮忙看一次 flow 为什么失败

你的动作应该是：

1. 运行 `flowctl status` 找失败 run。`
2. 打开 `runs/run_xxx/logs/flow.log`。`
3. 找第一个失败步骤。
4. 打开该步骤对应 report。
5. 检查 `manifest.json` 里的输入和 commit。`
6. 整理成一句清楚描述。

你应该这样汇报：

我检查了 run_xxx，失败发生在 precheck 步骤。flow.log 中第一个失败原因是缺少 README.md。我确认 design 目录下没有 README，建议先补齐 IP 交付文档后重新运行。

场景 2：导师问这次结果为什么和上次不一样

你的动作应该是：

```
flowctl compare run_001 run_002
cat runs/run_001/manifest.json
cat runs/run_002/manifest.json
```

你要判断：代码版本是否变了，输入文件 hash 是否变了，flow 配置是否变了，工具版本是否变了。

场景 3：导师给你一个新 IP 包，让你先检查一下

你的动作应该是：

```
flowctl qa --design 新IP目录名
```

然后打开 HTML QA 报告，把 FAIL/WARN 项整理出来。

场景 4：导师让你统计 testchip 失败原因

你的动作应该是看 `testchip_analysis.md/json`。`如果是真实数据，则需要确认 CSV 字段、筛选条件和统计口径。

场景 5：导师问你会不会用 AI 工具

不要只说会 ChatGPT。你可以说：我理解内部 AI 工具要先考虑数据权限和文档来源，本项目里用 `flowctl ask` 做了一个简单的本地文档检索，模拟内部知识库问答入口。`

第 19 章：代码伪代码和函数地图

这一章不用你背语法，而是用伪代码帮你理解程序结构。

flowctl run 的伪代码

```
读取命令参数 design 和 flow
找到 designs/<design> 目录
找到 config/<flow>_flow.yaml
读取 steps 列表
创建新的 runs/run_XXX 目录
写入 manifest.json，状态为 RUNNING
for step in steps:
    写日志：step start
    if step == precheck:
        检查输入文件
    elif step 是 synth/floorplan/place/route/signoff:
        调用 Tcl 脚本或 fallback
    elif step == ip_qa:
        检查 IP 交付包
    elif step == data_collect:
        分析 testchip CSV
    elif step == report:
        生成总报告
    记录 step PASS 和耗时
如果全部成功:
    manifest status = PASS
如果中途失败:
    manifest status = FAIL
    记录 error
```

函数地图

函数	作用	你应该理解到什么程度
cmd_doctor	检查环境	知道它检查 Python/Git/Tcl
cmd_run	主流程	必须理解整体逻辑
precheck	输入检查	知道缺文件会失败
run_tcl_step	执行 Tcl 步骤	知道有 tclsh 和 fallback 两条路径
run_ip_qa	IP QA	知道检查项和输出
analyze_testchip_data	测试数据分析	知道统计良率和失败分布
build_report	生成 HTML/Markdown 报告	知道它汇总 metrics/QA/data
cmd_compare	比较两次 run	知道它比较 metrics
cmd_ask	搜索文档	知道它是本地关键词检索
cmd_learn	显示学习路线	知道它是新手入口

Tcl 脚本的共同结构

读取命令行参数：

```
run_dir  
design_dir  
metrics_path  
report_path
```

设置模拟指标：

```
area / power / timing / drc / lvs
```

写 metrics 文件：

```
key=value
```

写 report 文件：

```
人能读懂的报告
```

真实 EDA 工具里，设置模拟指标的位置会变成实际 EDA 命令，例如

read_verilog、compile、place_design、route_design、report_timing、report_power 等。

附录 A：速查表

我要做什么	命令
看学习路线	<code>flowctl learn</code>
检查环境	<code>flowctl doctor</code>
跑完整流程	<code>flowctl run --design alu --flow testchip</code>
看 run 列表	<code>flowctl status</code>
看日志	<code>tail -n 80 runs/run_001/logs/flow.log</code>
单独做 QA	<code>flowctl qa --design alu</code>
比较两次结果	<code>flowctl compare run_001 run_002</code>
搜索文档	<code>flowctl ask "怎么看日志"</code>
归档结果	<code>flowctl archive --run latest</code>

附录 B：术语表

术语	解释
EDA	Electronic Design Automation，电子设计自动化工具
Flow	一串自动化步骤
RTL	寄存器传输级设计代码，常见 Verilog/SystemVerilog
SDC	时序约束文件
Synthesis	综合，把 RTL 转换为门级网表
Floorplan	版图规划
Place	标准单元摆放
Route	布线
Signoff	签核检查
DRC	设计规则检查
LVS	版图与原理图一致性检查
WNS	Worst Negative Slack，最差时序裕量
IP QA	IP 交付质量检查
Manifest	一次运行的版本和输入记录
Hash	文件指纹，用于判断文件是否变化